

0.29. Revenir au départ avec va_a

Le 0.28, plus une ligne : `va_a(x, y, ..., 0, 0)` ramène la lettre au coin, sans compter les pas du retour.

1. Ce qu'il faut comprendre

C'est le 0.28, plus une ligne : le retour à (0, 0).

Ce qui est nouveau ici : rien qu'une autre arrivée. Pour revenir, on ne calcule pas « -10 et -5 » : on dit simplement où : `0, 0`.

Le chemin du retour suit la même règle : en diagonale tant que les deux axes avancent, de (10, 5) à (5, 0), puis tout droit jusqu'en (0, 0).

Les fonctions de cette leçon — et le tuto de chacune

- `#include <va_a>` → [0.27.3. La fonction va_a\(\) — aller jusqu'à une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1176 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      62 o ← la console, à l'allumage - la mise en route de la console
va_a()          176 o ← main() - une fonction fournie par le langage
main()          83 o ← démarrage - ta fonction
VBlank          8 o  ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage   28 o ← image() - attend l'image suivante
AttendreVBlank  16 o ← démarrage - attend que l'écran soit libre
AttendreAcces   12 o ← EffacerCases, EcrireNombre - attend de pouvoir écrire une case
AdresseCase     10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() - efface des cases
EcrireNombre    33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles    18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte    14 o ← démarrage - vide les deux cartes de l'écran
Tuiles          704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser()  x 2    26 o sur place, dans va_a() + AdresseCase (10 o)
nombre() x 2    20 o sur place, dans main() + EcrireNombre (33 o)
effacer() x 1   15 o sur place, dans va_a() + AdresseCase (10 o), EffacerCases (12 o)
image()  x 2     6 o sur place, dans va_a(), main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
va_a()  x 2    44 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <va_a>          // mène une tuile jusqu'à une case
#include <ALPHABET>       // les lettres de la police : ALPHABET[0] est le A
#include <nombre>         // écrit un nombre en chiffres
```

2. UNE VARIABLE — X

```
uint8_t x = 0;          // la COLONNE de la lettre
```

3. UNE VARIABLE — Y

```
uint8_t y = 0;          // la LIGNE de la lettre
```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    va_a(x, y, ALPHABET[0], 10, 5);  // l'aller (le 0.28) : jusqu'en (10, 5)

    // La ligne ajoutée : l'arrivée est le coin (0, 0).
    // de (10, 5), en diagonale jusqu'en (5, 0), puis tout droit jusqu'en (0, 0)
    va_a(x, y, ALPHABET[0], 0, 0);

    nombre(0, 17, x);    // 000
    nombre(4, 17, y);    // 000

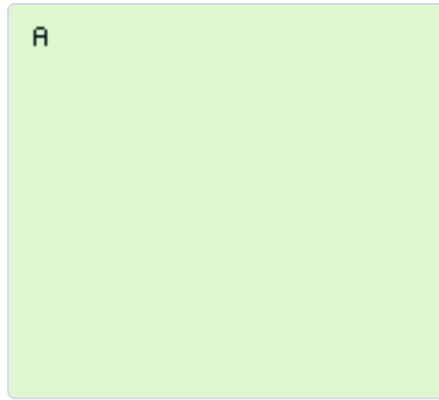
    while (true) {
        image();
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



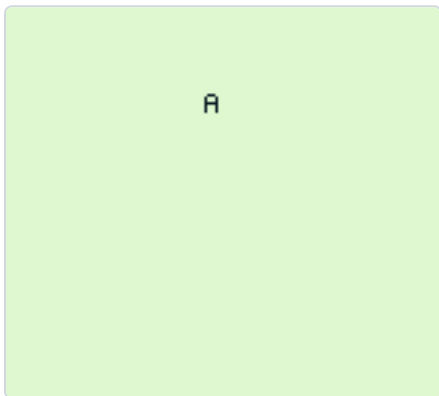
la toute première image — la console vient de démarrer — image 1



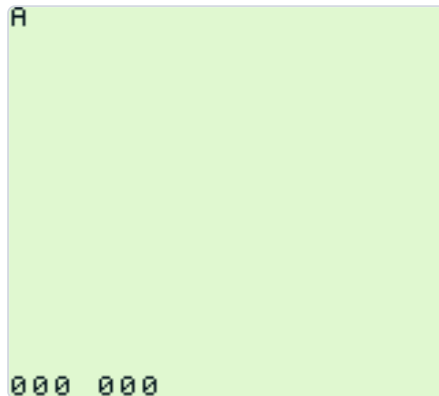
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A va jusqu'en (10, 5), puis revient au coin (0, 0), en biais puis tout droit.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ l'aller jusqu'en (10, 5)
- ✓ le retour en biais : (7, 2), puis tout droit : (3, 0)
- ✓ revenu en (0, 0) ; x et y valent 0

Ensuite : **0.29.1. Revenir au départ avec `va_a` — de base, ailleurs** — chapitre 0 : Avant tout. · `gameboy3` — écrire une cartouche Game Boy en C++